

# Séries temporais e sistemas de recomendação

Fabio Santiago
fabio.santiago@facens.br

Facens




# Sistema de recomendação FunkSVD

# Cronograma

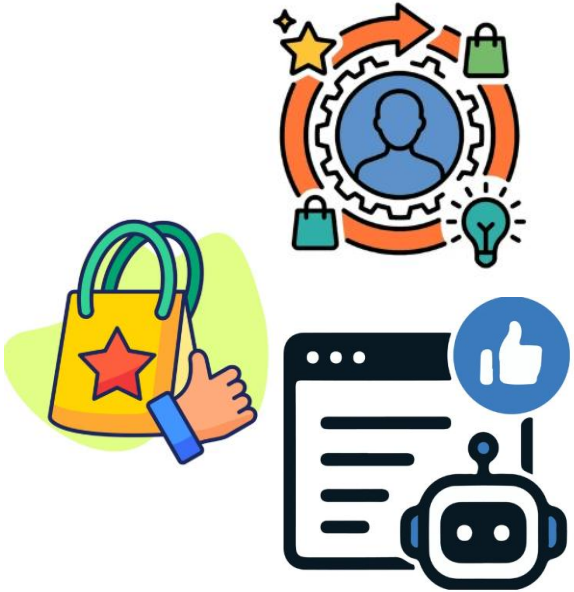
- O problema central
- Por que fatoração de matrizes?
- O desafio da esparsidade
- Filtragem colaborativa e fatoração de matrizes
- A ideia da fatoração
- O SVD tradicional e suas limitações
- A solução improvisada e ruim
- Funk SVD: a virada de paradigma
- Características do FunkSVD
- ALS (Alternating Least Squares)

# O problema central

**O Netflix Prize (2006–2009)**

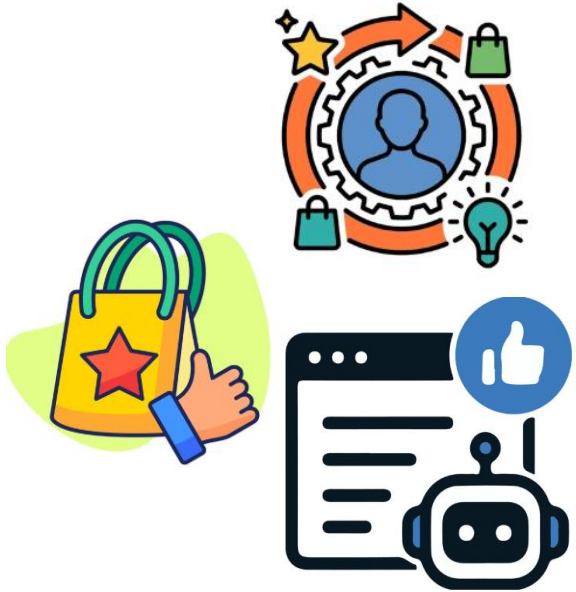
Sistemas de recomendação buscam prever o **interesse** de um usuário por itens que ele ainda não consumiu. A pergunta fundamental é:

**Dado um histórico de interações (avaliações, cliques, compras), como estimar a nota ou preferência para pares (usuário, item) não observados?**



# Por que fatoração de matrizes?

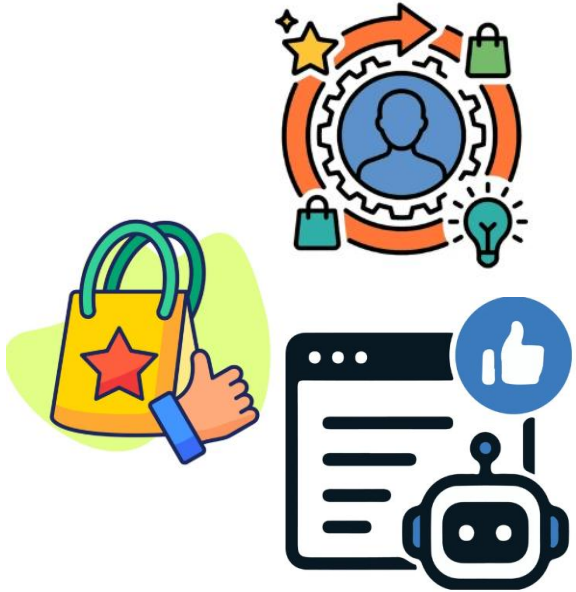
A **Filtragem Colaborativa (CF)** parte do princípio de que usuários com comportamentos semelhantes no passado tendem a ter preferências semelhantes no futuro. Dentro da CF, a **fatoração de matrizes** se tornou dominante por:



- Capturar **fatores latentes** (dimensões ocultas de gosto);
- Reduzir dimensionalidade e lidar com esparsidade;
- Escalar para milhões de usuários e itens;
- Ter bom desempenho empírico em competições e produção.

# O desafio da esparsidade

Em sistemas reais, a matriz de avaliações costuma ter **mais de 99% de valores ausentes**. Exemplo: 1,6 milhão de usuários × 16 mil itens = ~25,6 bilhões de células possíveis, mas apenas ~85 milhões observadas (0,3% de densidade).



# Filtragem colaborativa e fatoração de matrizes

**Duas grandes famílias**

| Família | Ideia central | Exemplos |
|---|---|---|
| Baseada em memória | Usa similaridade entre usuários ou itens | KNN, similaridade de cosseno |
| Baseada em modelo | Aprende um modelo preditivo a partir dos dados | ALS, FunkSVD, FM, NCF |

# A ideia da fatoração

A fatoração de matrizes assume que existe uma representação latente de baixa dimensão capaz de explicar as preferências:

$$\boldsymbol{R \approx X \cdot Y^T}$$

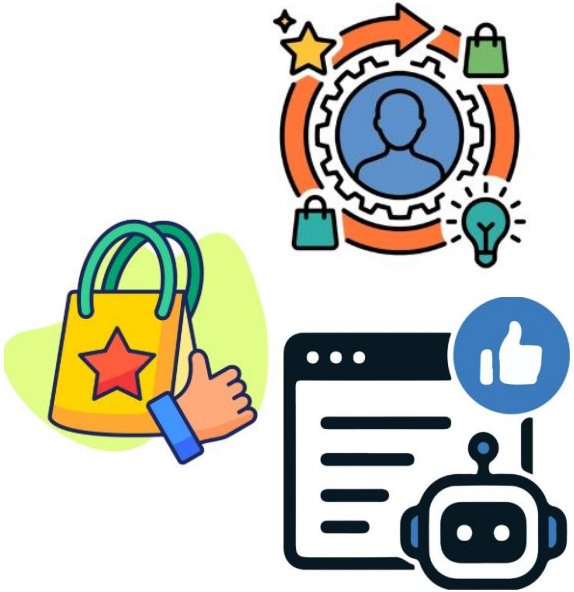
Sendo:

**$R$:** matriz usuário × item (esparsa);

**$X$:** matriz de fatores latentes dos usuários ($n_u \times k$);

**$Y$**: matriz de fatores latentes dos itens ($n_i \times k$);

**$k$**: número de dimensões latentes (tipicamente 10–200).



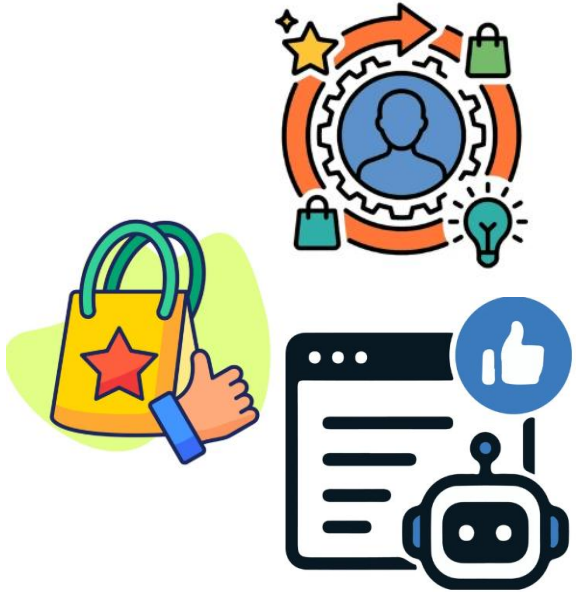
# O SVD tradicional e suas limitações

**O que é SVD**

A Decomposição em Valores Singulares fatora uma matriz $R$como:

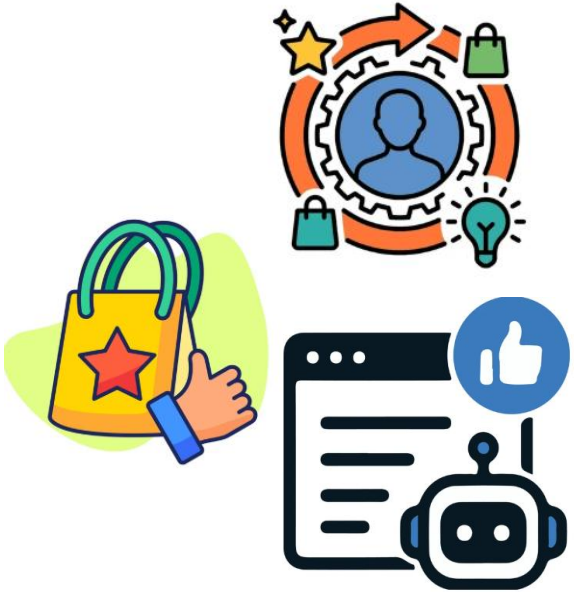
$$R = U\Sigma V^T$$

onde $U$e $V$são ortogonais e Σé diagonal com os valores singulares.



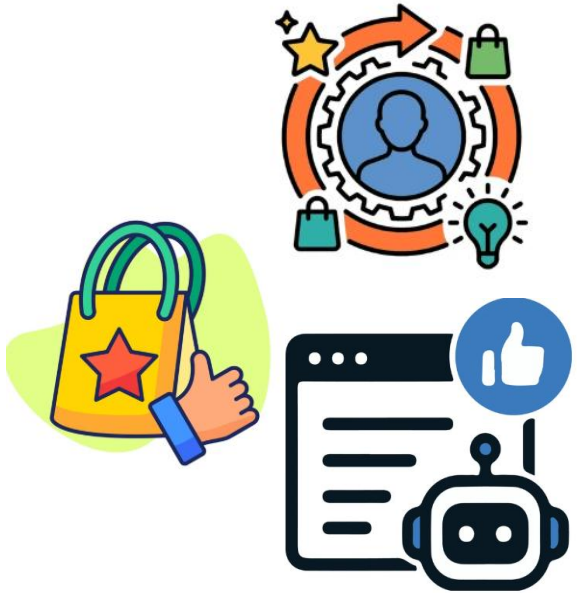
# Problemas no contexto de recomendação

| Problema | Descrição |
|---|---|
| Exige matriz completa | O SVD matemático não está definido para matrizes com valores ausentes |
| Preenchimento distorce | Preencher ausentes com média cria avaliações "neutras" artificiais |
| Custo computacional | SVD de matriz densa é caro e inviável em escala |
| Interpretação incorreta | A estrutura ortogonal não reflete a natureza dos dados de preferência |



# A solução improvisada e ruim

A abordagem clássica era: **imputar** os ausentes (média global, média do item, etc.) e então aplicar SVD. Isso introduz viés sistemático e compromete a qualidade das previsões.



# Funk SVD: a virada de paradigma

**A inovação**

Simon Funk, no Netflix Prize (2006), propôs algo radicalmente diferente:

**Não preencher os ausentes. Modelar apenas as avaliações observadas.**



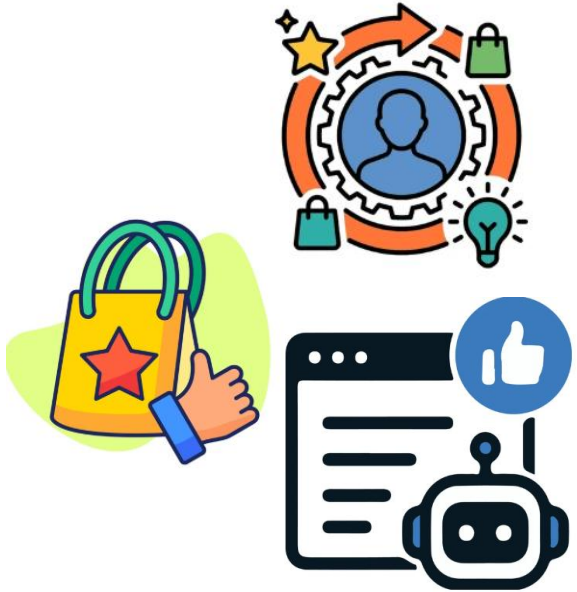
# Características do FunkSVD

- Decomposição $\hat{R} = P^T Q$ , **sem exigir ortogonalidade ou diagonalidade**;
- Função objetivo calculada **apenas sobre as entradas observadas**;
- Regularização L2 para evitar overfitting;
- Otimização via **SGD (Stochastic Gradient Descent)** ou ALS.



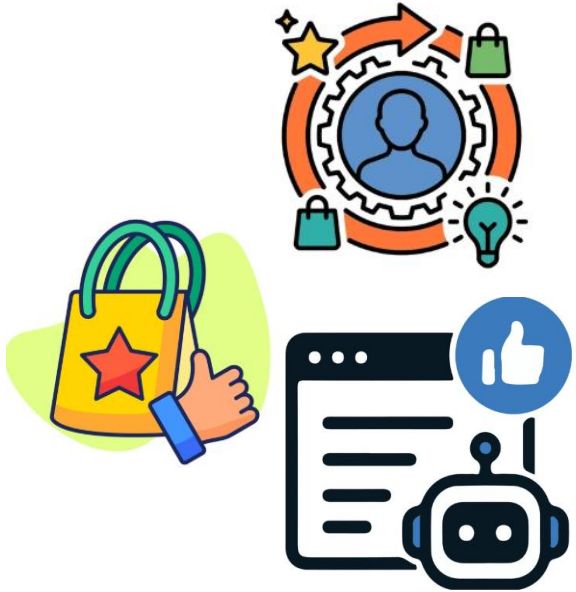
# Comparando SVD e FunkSVD

| Aspecto | SVD tradicional | FunkSVD |
|---|---|---|
| Dados ausentes | Preenchidos | Ignorados |
| Viés | Alto | Baixo |
| Escalabilidade | Limitada | Alta |
| Flexibilidade | Rígida | Alta |
| Base teórica | Álgebra linear | Otimização |



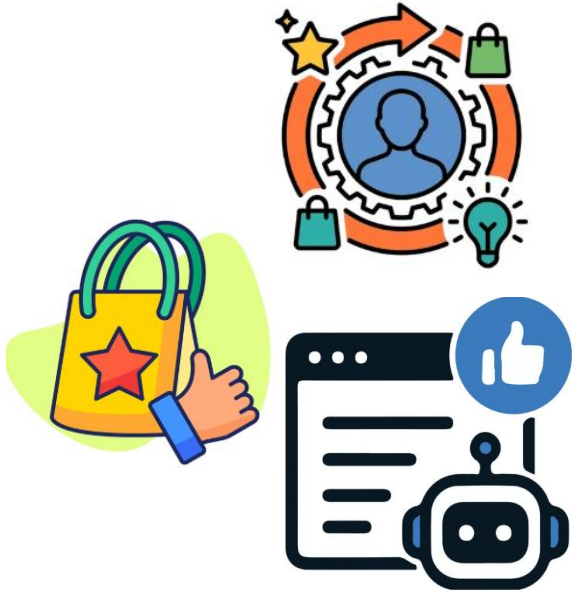
# ALS (Alternating Least Squares)

**ALS (Alternating Least Squares)** é um **algoritmo de otimização** para resolver o problema de fatoração de matrizes. Seu nome vem da estratégia de **alternar** entre a resolução de uma matriz e outra.



# Função objetivo

$$\min_{P,Q} \sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - p_u^T q_i\right)^2 + \lambda \left(\|p_u\|^2 + \|q_i\|^2\right)$$

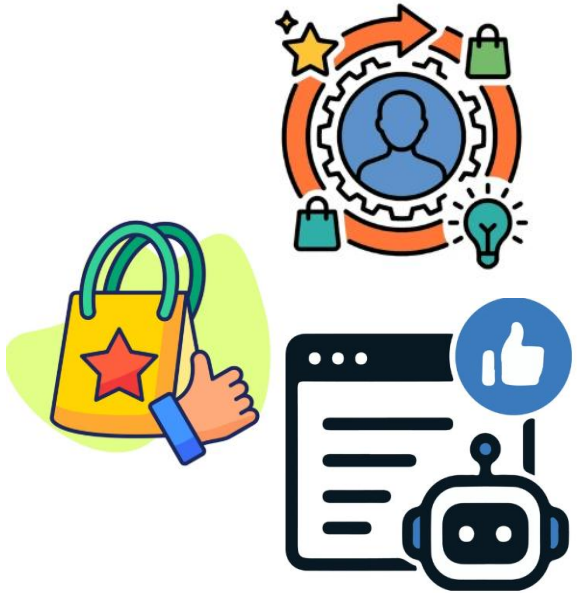


onde $\mathcal{O}$é o conjunto de pares observados e $\lambda$controla a regularização.

# ALS (Alternating Least Squares)

O ALS resolve o seguinte problema de otimização:

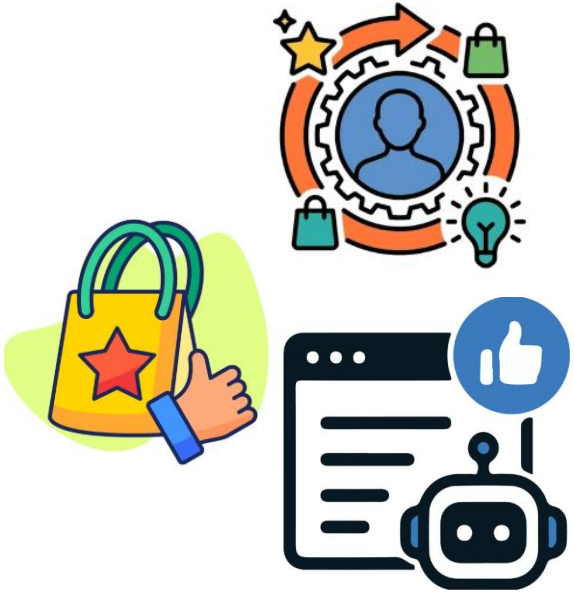
$$\min_{X,Y} \mathcal{L}(X,Y) = \sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - x_u^T y_i\right)^2 + \lambda \sum_u \|x_u\|^2 + \lambda \sum_i \|y_i\|^2$$



# ALS (Alternating Least Squares)

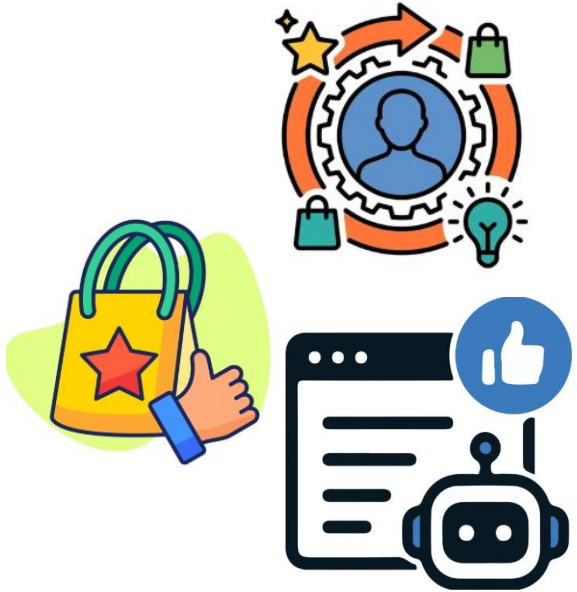
Sendo

- $\mathcal{O}$é o conjunto de pares (usuário, item) **observados**;
- $x_u \in \mathbb{R}^k$é o vetor de fatores latentes do usuário $u$;
- $y_i \in \mathbb{R}^k$é o vetor de fatores latentes do item $i$;
- $\lambda \geq 0$é o parâmetro de regularização L2;
- $r_{ui}$é a nota observada.



# ALS (Alternating Least Squares)

O ponto-chave do ALS é que essa função **não é convexa em** $(X, Y)$**conjuntamente**, mas é **convexa em** $X$**quando** $Y$**é fixado** e vice-versa. Isso é o que permite a solução fechada

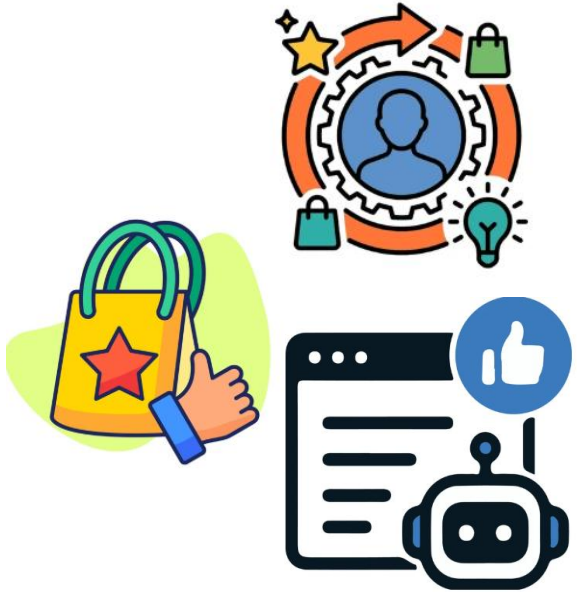


# ALS (Alternating Least Squares)

**Caso 1 : Fixando X , Otimizando Y**

Fixamos $X$. O termo $\lambda \sum_u \| x_u \|^2$ passa a ser **constante** (não depende de $Y$) e pode ser removido do argmin:

$$\min_Y \mathcal{L}_Y(Y) = \sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - x_u^T y_i\right)^2 + \lambda \sum_{i=1}^{n_i} \|y_i\|_2^2$$



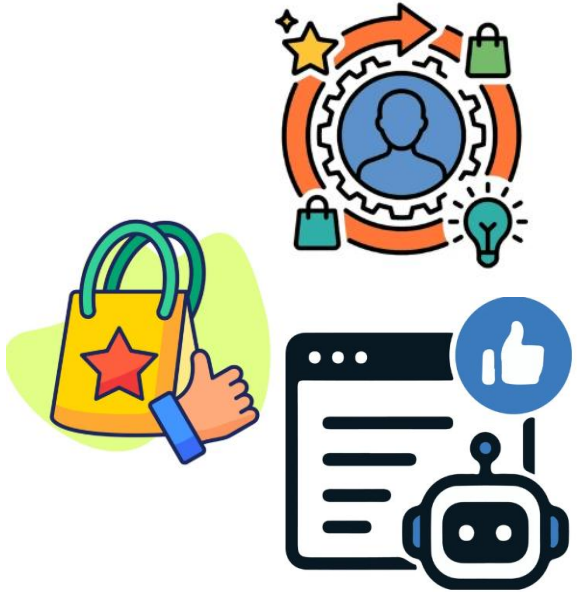
# ALS (Alternating Least Squares)

**Caso 1 : Fixando X , Otimizando Y**

A soma sobre $(u,i) \in \mathcal{O}$pode ser reorganizada agrupando por item

$$\sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - x_u^T y_i\right)^2 = \sum_{i=1}^{n_i} \sum_{u\in\mathcal{U}_i} \left(r_{ui} - x_u^T y_i\right)^2$$

onde $\mathcal{U}_i = \{u: (u,i) \in \mathcal{O}\}$é o conjunto de usuários que avaliaram o item $i$

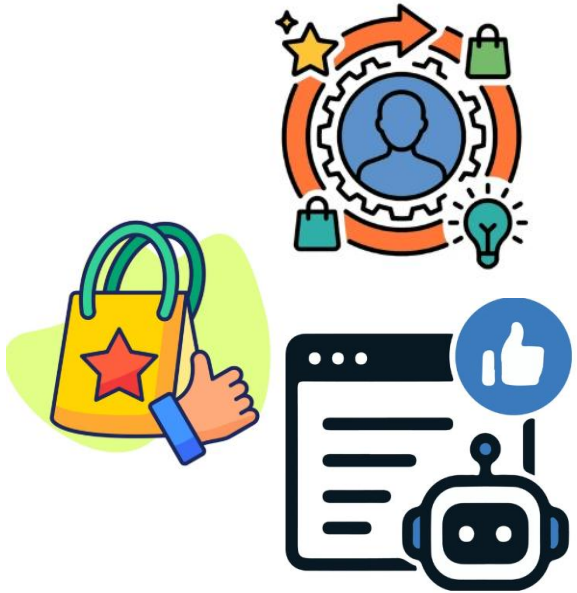


# ALS (Alternating Least Squares)

**Caso : Fixando X , Otimizando Y**

Como o termo de regularização também é separável por item, o problema **se desacopla em $n_i$ subproblemas independentes**:

$$\min_{y_i} \mathcal{L}_i(y_i) = \sum_{u \in \mathcal{U}_i} \left(r_{ui} - x_u^T y_i\right)^2 + \lambda \|y_i\|_2^2, \quad i = 1, \ldots, n_i$$



# ALS (Alternating Least Squares)

**Caso 1 : Fixando X , Otimizando Y**

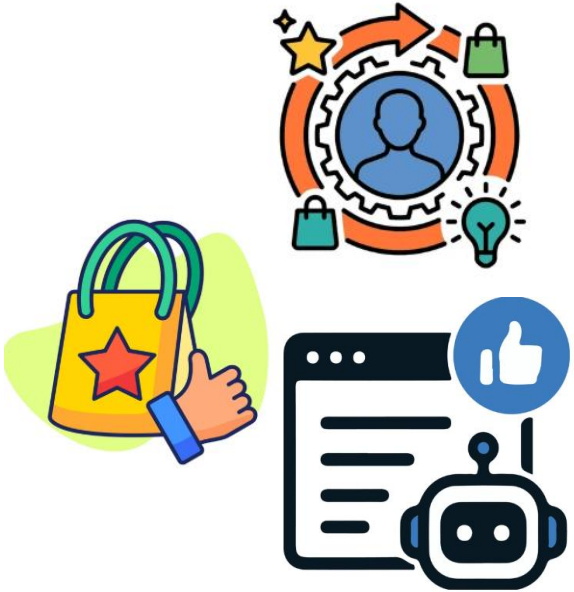
FORMA MATRCIAL

Definimos:

- $X_i \in \mathbb{R}^{|\mathcal{U}_i| \times k}$: matriz cujas linhas são $x_u^T$ para $u \in \mathcal{U}_i$;
- $r_i \in \mathbb{R}^{|\mathcal{U}_i|}$: vetor de notas observadas $r_{ui}$ para $u \in \mathcal{U}_i$.

O subproblema se escreve como:

$$\min_{y_i} \mathcal{L}_i(y_i) = \| r_i - X_i\, y_i \|_2^2 + \lambda \| y_i \|_2^2$$



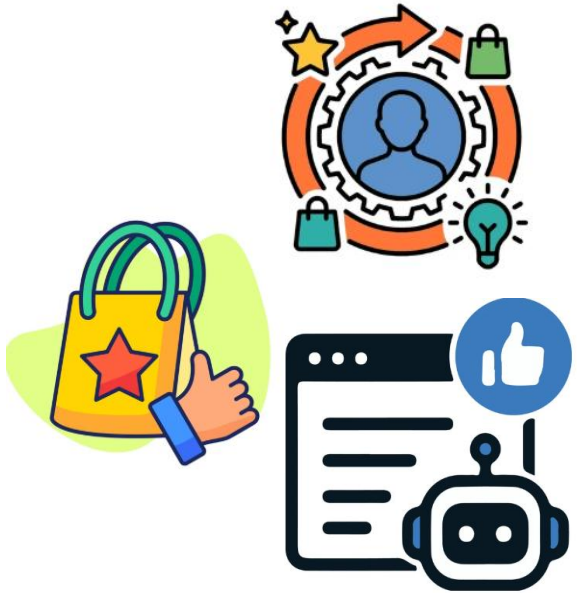
# ALS (Alternating Least Squares)

**Caso 1 : Fixando X , Otimizando Y**

Solução fechada.

Como $X_i^T X_i + \lambda I_k$ é **simétrica positiva definida** (garantido por $\lambda > 0$), ela é invertível:

$$y_i = \left(X_i^T X_i + \lambda I_k\right)^{-1} X_i^T r_i$$

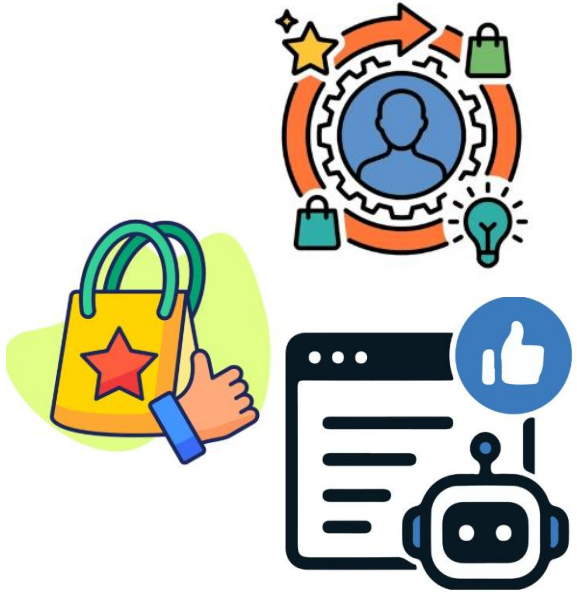


# ALS (Alternating Least Squares)

**Caso 2: Fixando $Y$, Otimizando $X$**

Fixamos $Y$. O termo $\lambda \sum_i \| y_i \|^2$ é constante e pode ser removido:

$$\min_X \mathcal{L}_X(X) = \sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - x_u^T y_i\right)^2 + \lambda \sum_{u=1}^{n_u} \|x_u\|_2^2$$



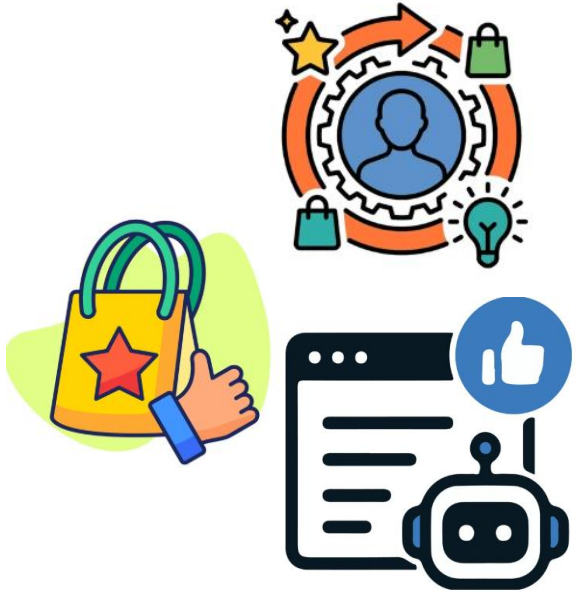
# ALS (Alternating Least Squares)

**Caso 2: Fixando $Y$, Otimizando $X$**

**Decomposição por usuário**

Reorganizando a soma agrupando por usuário:

$$\sum_{(u,i)\in\mathcal{O}} \left(r_{ui} - x_u^T y_i\right)^2 = \sum_{u=1}^{n_u} \sum_{i\in\mathcal{I}_u} \left(r_{ui} - x_u^T y_i\right)^2$$

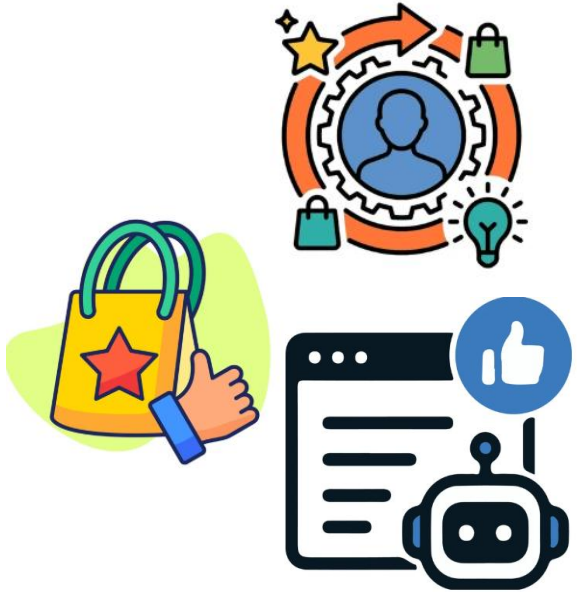


# ALS (Alternating Least Squares)

**Caso 2: Fixando $Y$, Otimizando $X$**

O problema **se desacopla em $n_u$subproblemas independentes**

$$\min_{x_u} \mathcal{L}_u(x_u) = \sum_{i \in \mathcal{I}_u} \left(r_{ui} - x_u^T y_i\right)^2 + \lambda \|x_u\|_2^2, \quad u = 1, \ldots, n_u$$



# ALS (Alternating Least Squares)

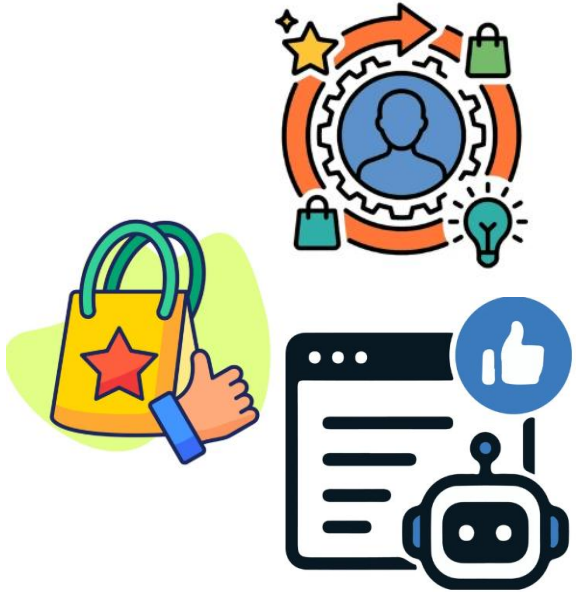
**Caso 2: Fixando $Y$, Otimizando $X$**

**Forma matricial**

Definimos:

$Y_u \in \mathbb{R}^{|\mathcal{I}_u| \times k}$: matriz cujas linhas são $y_i^T$ para $i \in \mathcal{I}_u$;

$r_u \in \mathbb{R}^{|\mathcal{I}_u|}$: vetor de notas observadas $r_{ui}$ para $i \in \mathcal{I}_u$



O subproblema se escreve como:

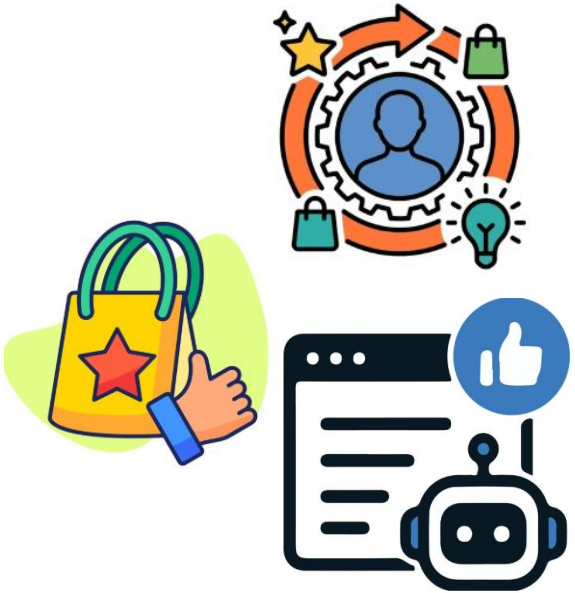
$$\min_{x_u} \mathcal{L}_u(x_u) = \|r_u - Y_u x_u\|_2^2 + \lambda \|x_u\|_2^2$$

# ALS (Alternating Least Squares)

**Caso 2: Fix1ando $Y$, Otimizando $X$**

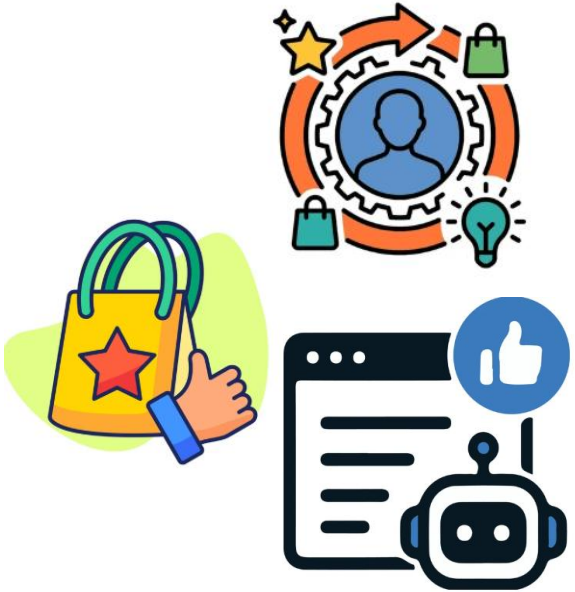
**Solução fechada**

$$x_u = \left(Y_u^T Y_u + \lambda I_k\right)^{-1} Y_u^T r_u$$



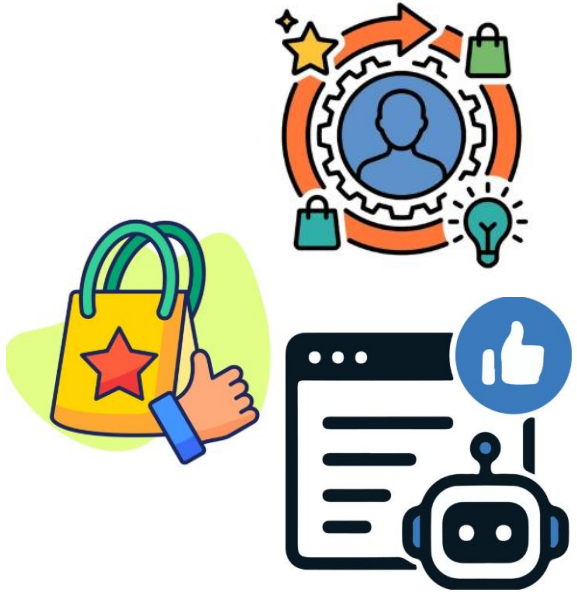
# Paralelização: o grande diferencial

- O cálculo de $x_u$ **não depende** de $x_{u'}$ para $u' \neq u$;
- O cálculo de $y_i$ **não depende** de $y_{i'}$ para $i' \neq i$;
- Portanto, cada linha pode ser resolvida em paralelo.



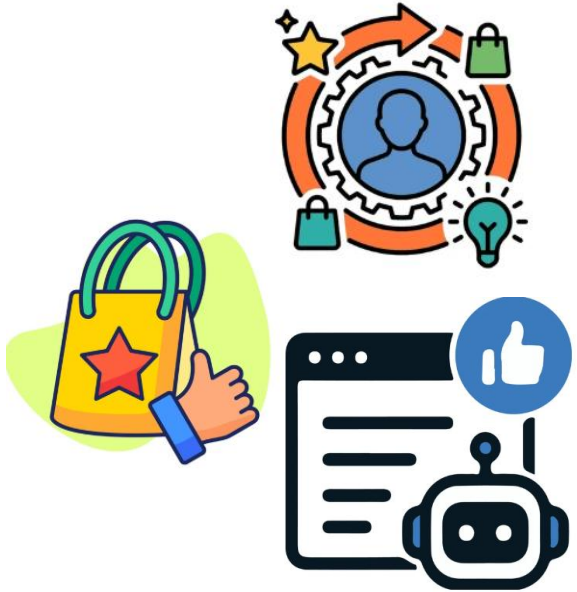
# Critérios de convergência

- Número fixo de iterações (mais comum em produção);
- Variação do erro abaixo de um limiar;
- Tempo máximo de execução.



# Sensibilidade a hiperparâmetros

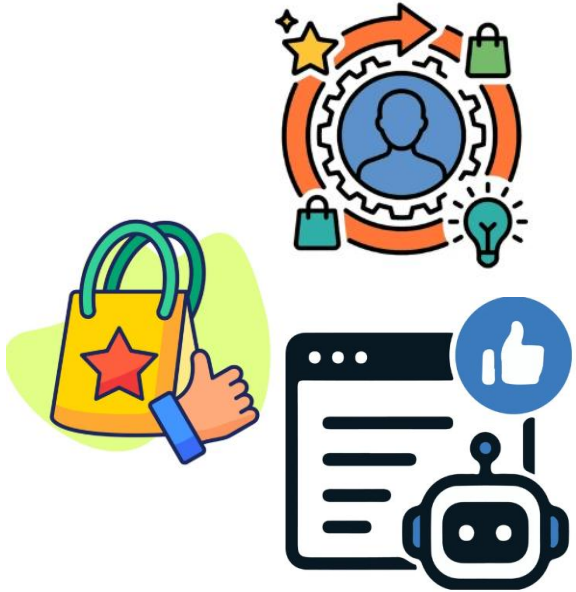
| Hiperparâmetro | Efeito |
|---|---|
| $k$(fatores latentes) | Mais fatores = mais expressividade, maior risco de overfitting |
| $\lambda$(regularização) | Controla overfitting; valores altos suavizam demais |
| Nº de iterações | Mais iterações = melhor ajuste, maior custo |
| $\alpha$(implícito) | Peso da confiança em feedback implícito |



# Vantagens ALS

**Vantagens:**

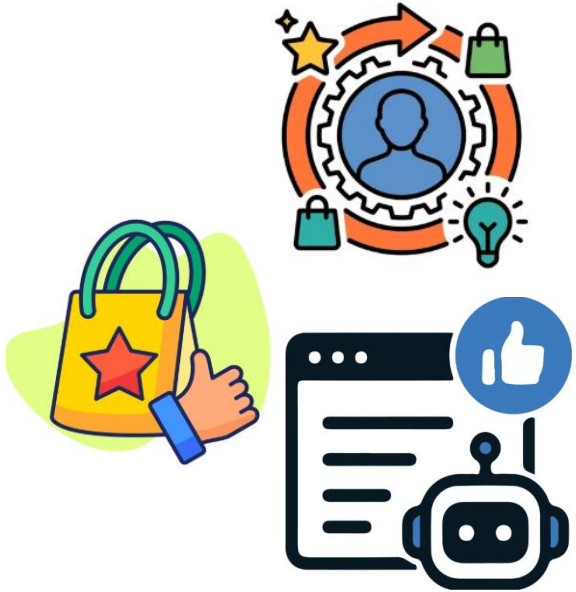
- Paralelização natural e eficiente;
- Sem taxa de aprendizado (solução fechada);
- Suporte maduro a feedback implícito;
- Validado em escala industrial (Spark MLlib);
- Bom desempenho em dados esparsos.



# Desvantagens ALS

**Desvantagens:**

- Custo elevado por iteração;
- Alto consumo de memória;
- Feedback implícito exige variante específica;
- Cold start problemático;
- Baixa interpretabilidade.



# Comparação entre os algoritmos de recomendação

| Dimensão | ALS | FunkSVD (SGD) | KNN |
|---|---|---|---|
| **Paradigma** | Fatoração + mín. quadrados | Fatoração + SGD | Similaridade |
| **Feedback implícito** | Sim (ponderado) | Não diretamente | Adaptável |
| **Paralelização** | Excelente | Ruim | Média |
| **Cold start** | Ruim | Ruim | Ruim |
| **Interpretabilidade** | Baixa | Baixa | Boa |
| **Escala** | Excelente | Boa | Limitada |

# Sistemas de recomendação

Implementando .....



# Sistemas de recomendação

**BIBLIOGRAFIA BÁSICA**

HYNDMAN, R. J.; ATHANASOPOULOS, G. Forecasting: Principles and Practice. 3. ed. Melbourne: OTexts, 2021.

BOX, G. E. P.; JENKINS, G. M.; REINSEL, G. C.; LJUNG, G. M. Time Series Analysis: Forecasting and Control. 5. ed. Hoboken: Wiley, 2015.

AGGARWAL, C. C. Recommender Systems: The Textbook. Cham: Springer, 2016. 6.

**BIBLIOGRAFIA COMPLEMENTAR**

RICCI, F.; ROKACH, L.; SHAPIRA, B. Recommender Systems Handbook. 2. ed. Cham: Springer, 2022. • GRUS, J. Data Science do Zero. 2. ed. Rio de Janeiro: Alta Books, 2020.

ALMEIDA, J. M. et al. Sistemas de Recomendação. Rio de Janeiro: LTC, 2016. • MORETTIN, P. A.; TOLOI, C. M. C. Análise de Séries Temporais. 3. ed. São Paulo: Blucher, 2018.

BROWNLEE, J. Deep Learning for Time Series Forecasting. Melbourne: Machine Learning Mastery, 2018.


Facens
AQUI TEM ENGENHARIA